

Peripherals and Interfacing

Peripherals: Peripherals are any device (internal or external) that connects to a computer to extend its capabilities.
attached to a CPU

3 Types :

① Input peripheral (Keyboard, Mouse, Scanner)

② Output Peripheral (Monitor, Printer, Speaker)

③ Memory Peripheral (RAM, ROM, storage)

Interface: Interface is the combination of hardware and software needed to link the CPU to the peripherals and to enable them to communicate with the CPU despite all their differing characteristics.

Peripheral characteristics

→ Mem Apu Note/slide

Function of peripheral

→ " " " "

Difference between Peripherals and interfacing:

Peripherals

Interface

External devices connected to the computer

Medium that connects and enable communication between CPU and peripheral

Expands computer functionality

Facilitate interaction between peripherals and the CPU

Example: Keyboard, mouse, printer

Example: USB, ~~mem~~, Bluetooth, HDMI

Buffer and Spooler

Buffer: A buffer is an area of ^{main} memory used which temporarily store data while transferring the data between the processor and peripheral.

Spoolers: Spoolers is a system program that ^{temporarily} stores data from input/output processes in secondary memory and allowing multiple jobs to be queued and process effectively ^{tasks}

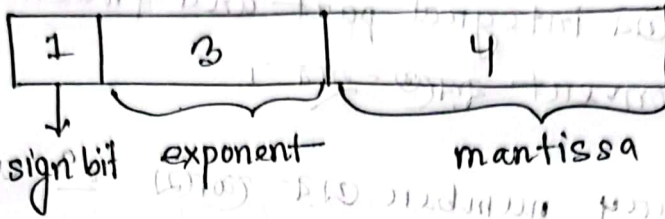
Q: Comparison between Buffer and Spoolers (2)

Buffer	Spoolers
1. Temporary storage for data during transfer	1. Manages and queues tasks for sequential processing
2. Handle speed mismatch between send CPU and Peripherals	2. Allows multiple tasks to be processed effectively
3. Holds data temporarily ^{in main memory} until the receiver is ready	3. Stores tasks in a queue in secondary memory and processes them one by one
Used in memory, I/O operations, streaming etc	Commonly used in printing, batch processing
Keyboard buffer, disk buffer	Print spooler, email queue

Floating Point Number

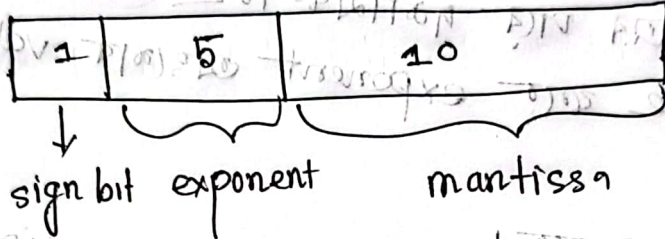
IEEE Floating point format

8 bit



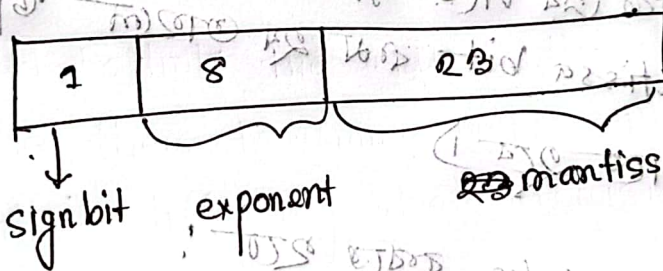
$k=3$

16 bit



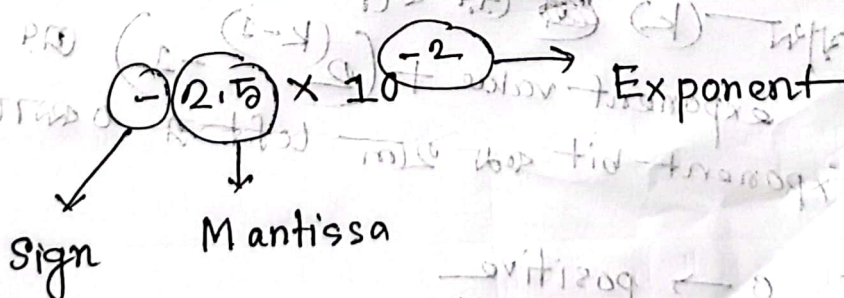
$k=5$

32 bit



$k=8$

For decimal number :



এই decimal number binary to convert করার পরে কিভাবে present হবে সেটার জন্য Floating point number represent in Binary.

* Memory I/O Regularly introducing a big overtake together for a regularship here use a memory management for regular operational theory management for our observation.

Main characteristics of interface:

- ① Have different data transfer rates
- ② Use a wide variety of codes and control signals

~~① Have different~~

- ③ Transmit data in serial or in parallel form.
- ④ Even work at higher voltages than the CPU
- ⑤ All operate at much slower speeds than the CPU.

Main functions of interface:

- ① Buffering
- ② Converting data to and from serial and parallel forms.
- ③ Converting data to and from analogue and digital forms
- ④ Voltage conversion
- ⑤ Protocol conversion
- ⑥ Handling of status signals:

Sarna Nam $\rightarrow$ Memory Interfacing

Problem-1: Suppose, a processor is having 16 bits address line and it is byte addressable. Interface 4096x8 RAM with the processor.

- ① What is the memory capacity of the processor?
- ② How many address lines are required for a RAM?
- ③ How many RAM chips are required.
- ④ What will be the size of address decoder?

Solution:

Given that,

Address line = 16

Word size = byte addressable = 8 bit

Interfacing 4096x8 = RAM size

$$\begin{aligned}\text{① Memory capacity of processor} &= \text{Address space} \times \text{word size} \\ &= 2^{\text{Address line}} \times \text{word size} \\ &= 2^{16} \times 8 \text{ bits} \\ &= 2^{16} \times 1 \text{ byte} \\ &= 2^6 \text{ K byte} \\ &= 64 \text{ KB}\end{aligned}$$

[Note: Address Space = RAM Address space / RAM capacity]

$$\textcircled{2} \text{ RAM Address Space} = 2^{\text{Address line}}$$

$$= 2^{16}$$

$$\therefore \text{Address line for a RAM} = \log_2 (\text{RAM Address space})$$

$$= \log_2 (2^{16})$$

$$= 16$$

$$\textcircled{2} \text{ Address line for a RAM} = \log_2 (\text{RAM locations})$$

$$= \log_2 (4096)$$

$$= \log_2 (2^{12})$$

$$= 12$$

$$\text{RAM Capacity} = 4096 = 2^{12}$$

$$\textcircled{3} \text{ Number of RAM chips required} = \frac{\text{Memory capacity}}{\text{RAM capacity}}$$

$$= \frac{2^{16}}{2^{12}}$$

$$= 2^4$$

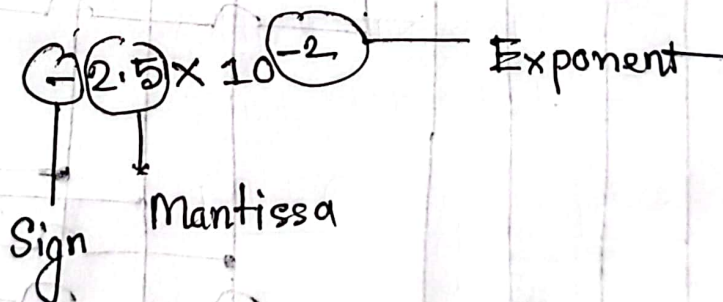
$$= 16$$

Size of decoder : $(16-12) \rightarrow 2^{(16-12)}$
 $= 4 \rightarrow 2^4$
 $= 4 \rightarrow 16 \text{ decoder}$

Floating point number

Floating point number contains 3 components

- ① Sign
- ② Exponent
- ③ Mantissa



Conversion of Floating point number to Binary

Main memory Array Design

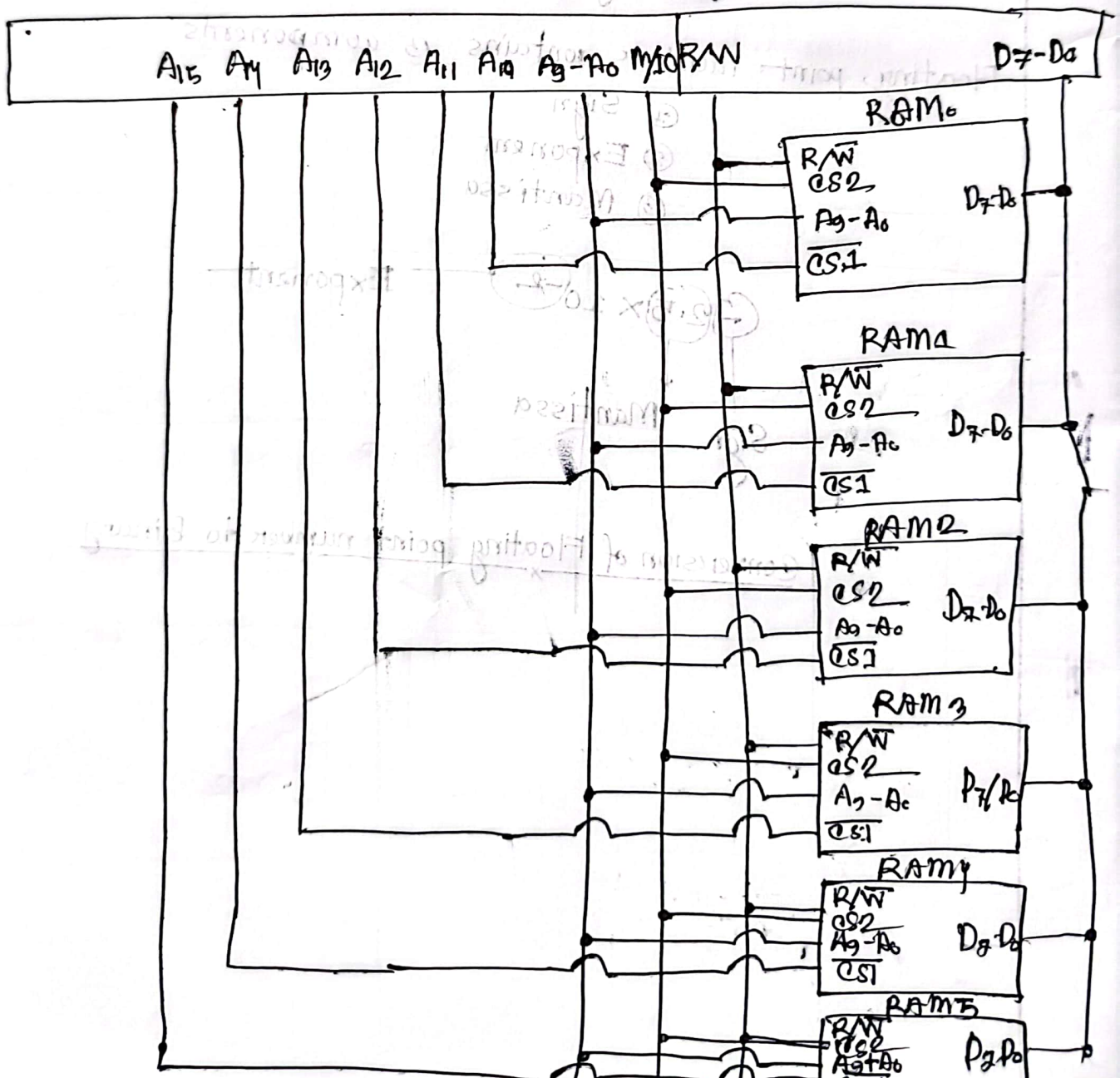
Microprocessor information like

Address bus
Data bus

RAM information

Address bus
Data bus

काठमाडौं RAM chip मासि



CHEMISTRY

ভৈষিক

* মেমোরি ম্যাপিংয়ের প্রকার - অসম্পূর্ণ address নেই

* memory mapping

Binary Address																Device	Address in Hex
A ₁₅	A ₁₄	A ₁₃	A ₁₂	A ₁₁	A ₁₀	A ₉	A ₈	A ₇	A ₆	A ₅	A ₄	A ₃	A ₂	A ₁	A ₀		
0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	RAM CHIP 0	0400
0	0	0	0	0	1	1	1	1	1	1	1	1	1	1	1		07FF
0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	RAM CHIP 1	0800
0	0	0	0	1	0	1	1	1	1	1	1	1	1	1	1		0BFF

Example 2: Interface two chips of 4K RAM and 1 chip of 2K ROM. Starting address of RAM is 8000H.

Solution:

Given that

RAM Capacity is 4K

$\therefore$ RAM Address bus = 12 bit

ROM capacity is 2K

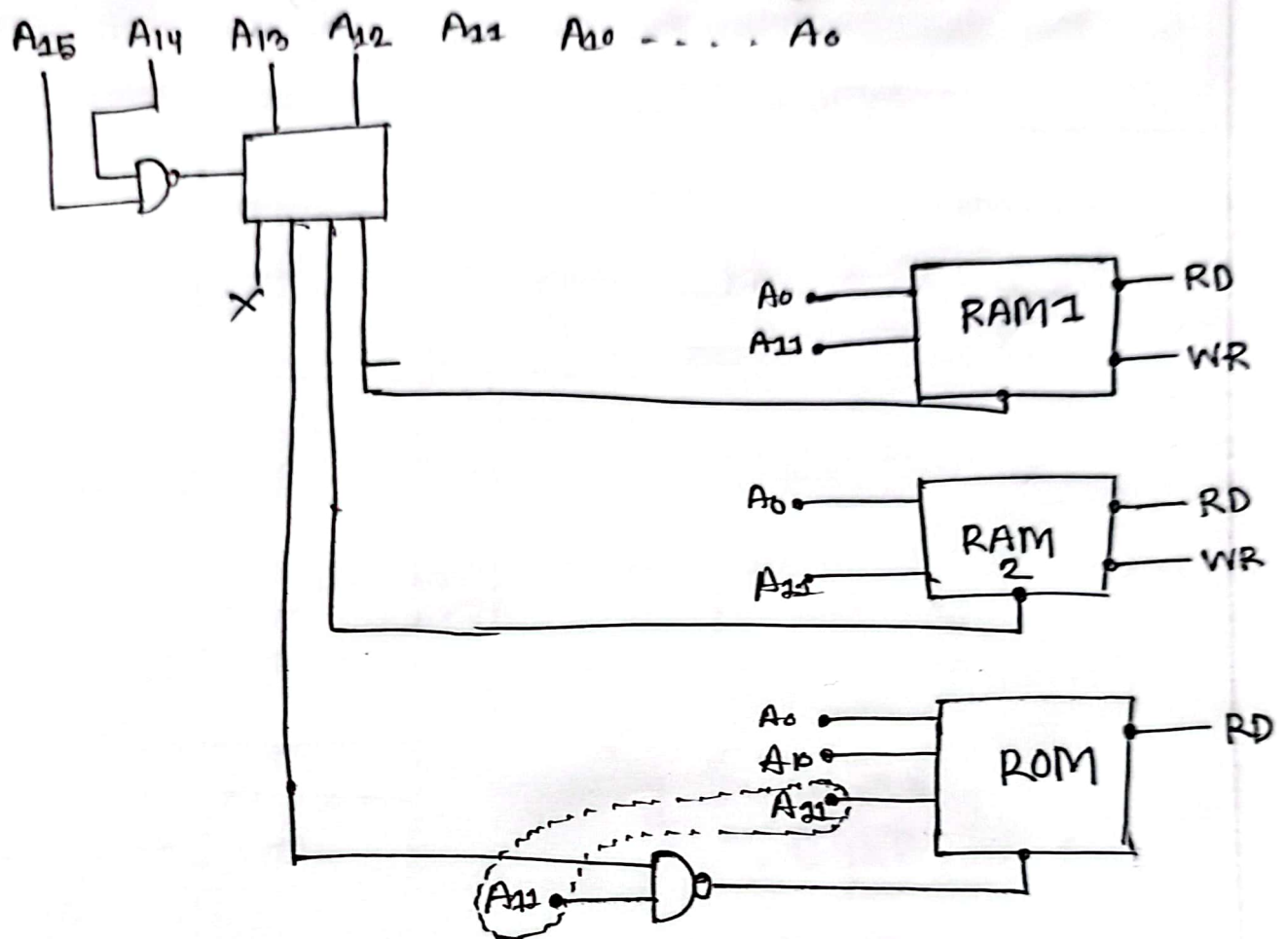
$\therefore$ ROM Address bus = 11 bit

The starting address of RAM is 8000H

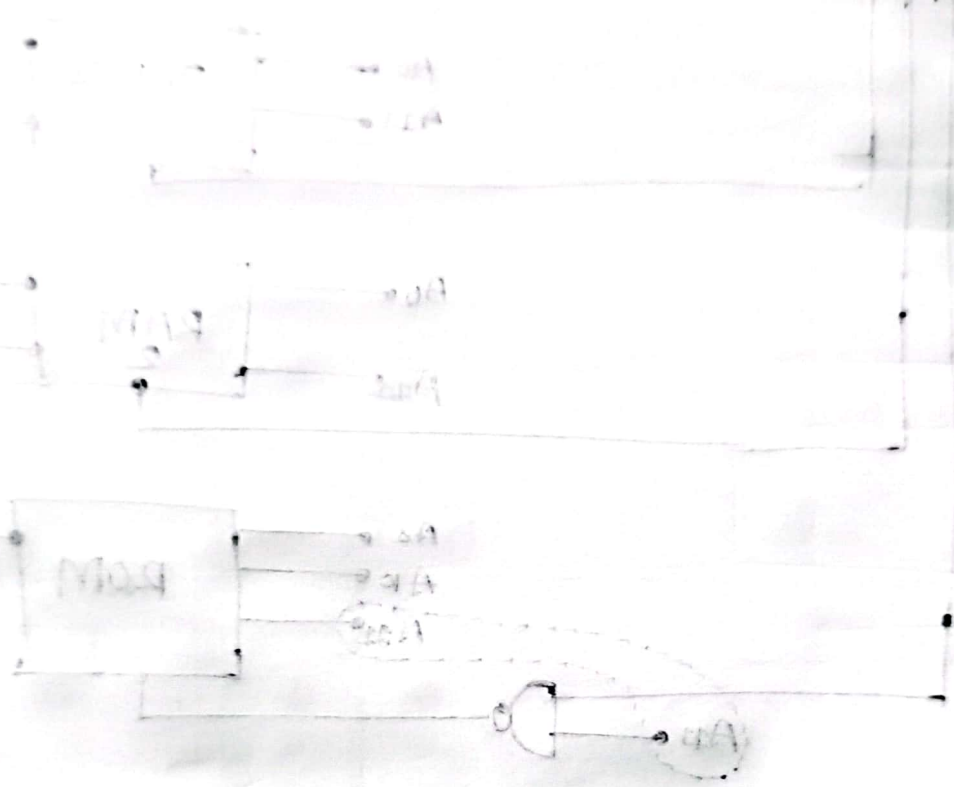
The memory mapping:

	A ₁₅	A ₁₄	A ₁₃	A ₁₂	A ₁₁	...	A ₀	
RAM1	1	0	0	0	0	...	0	8000H
	1	0	0	0	1	1	1	8FFFH
RAM2	1	0	0	1	0	...	0	9000H
	1	0	0	1	1	...	1	9FFFH
ROM	1	0	1	0	0/1	0	0	A000H - A0FFH (For 0)
	1	0	1	0	0/1	1	1	A800H - AFFFH (for 1)

∴ The Memory Array Diagram :



Disadvantages of linear decoding



5(a) From the given memory

Processor's Address bus = 16 bit
Data bus $\Rightarrow$ 8 bit

For RAM, Address bus = 10 bit
Data bus = 8 bit

Memory map : 0000_{16} to $0FFF_{16}$

$\therefore$ So the decoding technique is : Full decoding.

$$\text{Processor's capacity} = 2^{16} \times 8 \\ = 64 \text{KB}$$

$$\text{RAM capacity} = 2^{10} \times 8 \\ = 1 \text{KB}$$

$$\text{From memory map} = 0FFF - 0000 \\ = 0FFF$$

$$= 0 \times 16^2 + 15 \times 16^1 + 15 \times 16^0 + 15 \times 16^0 \\ = 4096 \text{ locations}$$

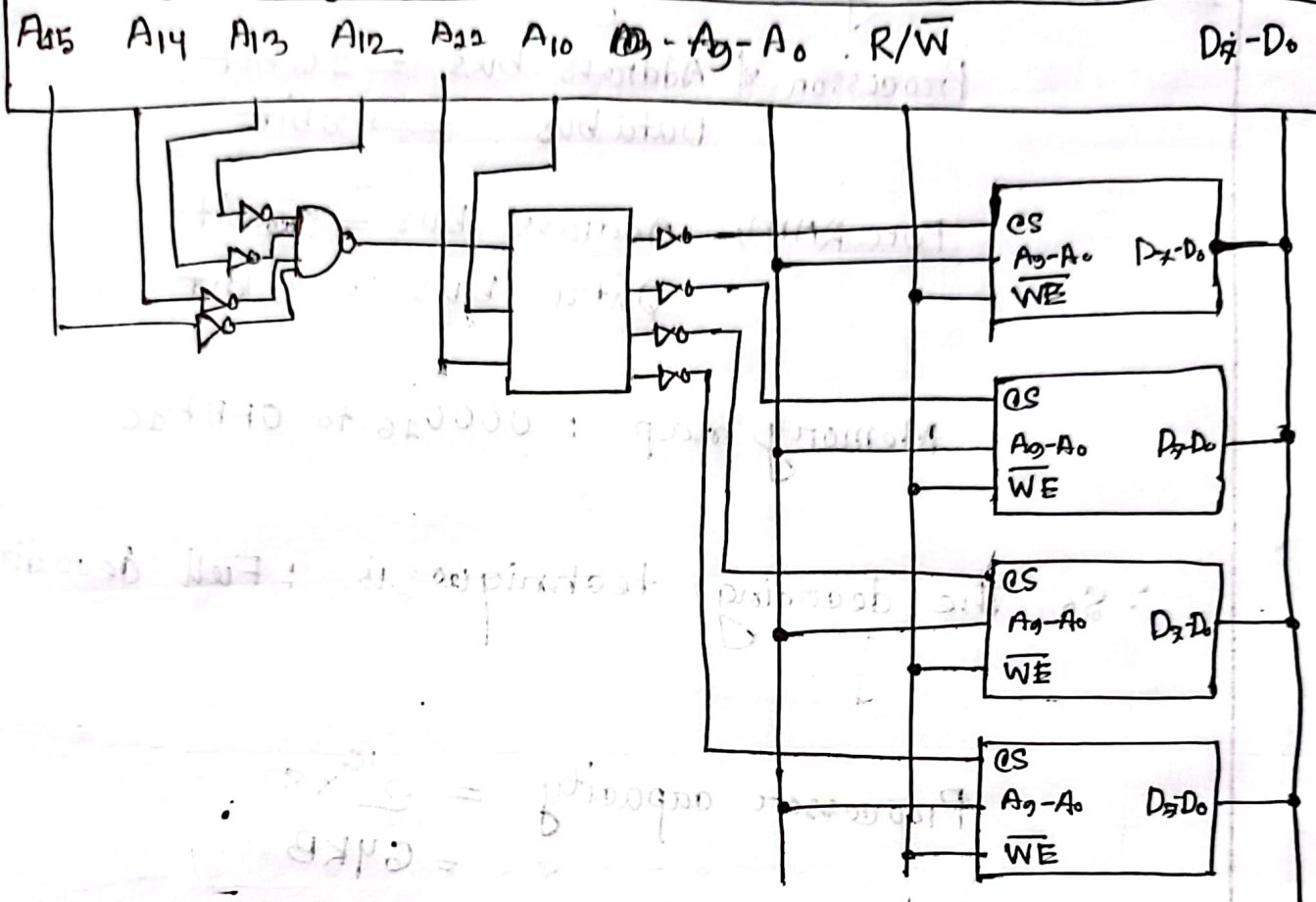
$$\text{Total memory locations} = 4096$$

$$\text{Each RAM has} = 1024$$

$\therefore$ We need ~~4 RAM~~ 1KB RAM

$$\frac{64 \text{KB}}{1 \text{KB}} =$$

The logic diagram for circuit



The memory Mapping :

A ₁₅	A ₁₄	A ₁₃	A ₁₂	A ₁₁	A ₁₀	A ₉	A ₈	A ₇	A ₆	A ₅	A ₄	A ₃	A ₂	A ₁	A ₀	Chip	Address Range
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0000	0000H
0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	00FF	03FFH
0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0100	0400H
0	0	0	0	0	1	1	0	0	0	0	0	0	0	0	0	01FF	07FFH
0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0200	0800H
0	0	0	0	1	0	1	1	1	1	1	1	1	1	1	1	02FF	0BFFH
0	0	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0300	0C00H
0	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	03FF	0FFFH

2026

64KB RAM
4KB RAM

5Q Given that

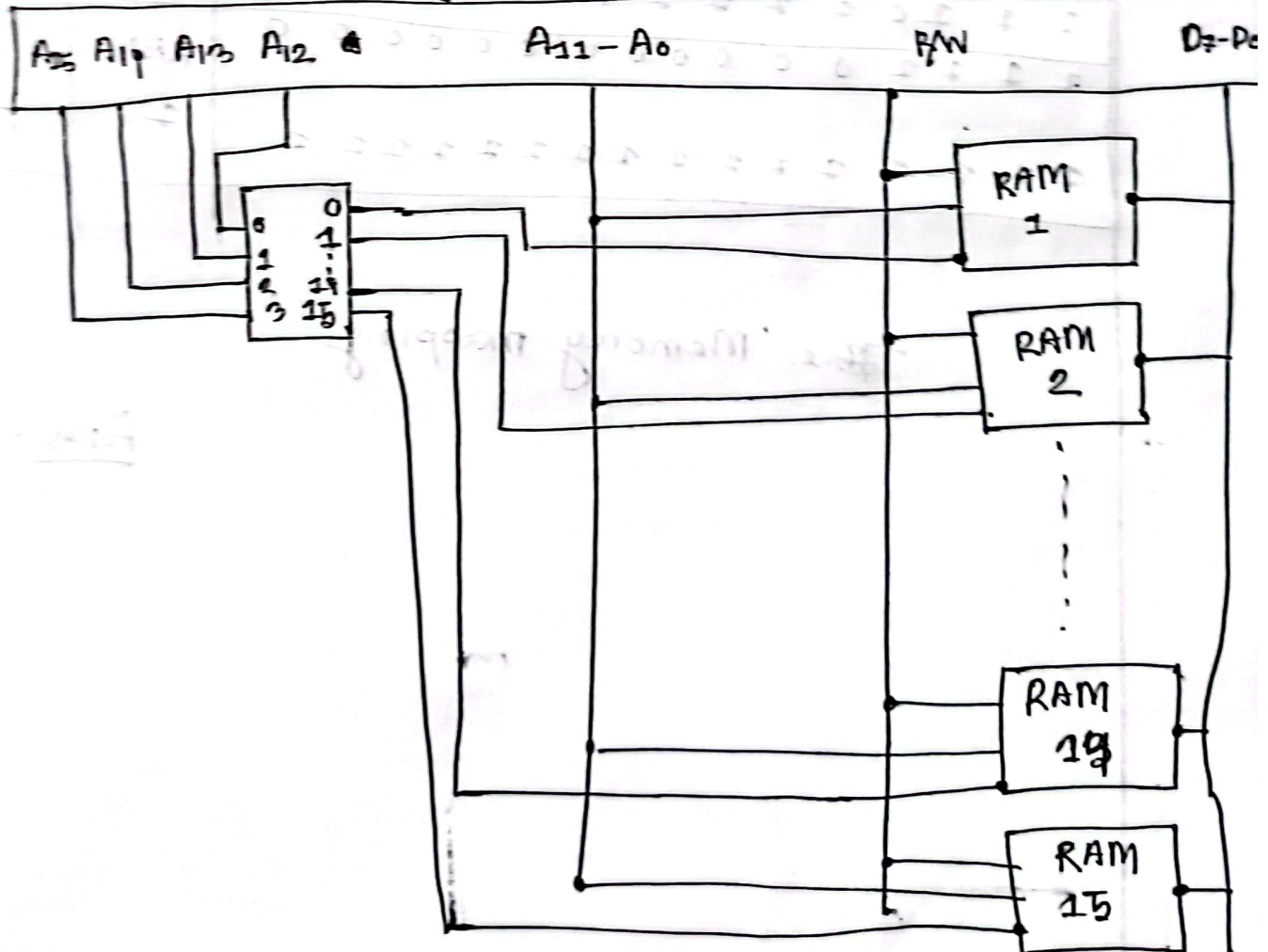
8-bit microprocessor : Address bus = 16 bit

$$\therefore \text{Processor capacity} = 2^{16} \times 8 = 64KB$$

RAM capacity = 4KB
 $\therefore$ RAM Address line = 12 bit

$$\therefore \text{Number of RAM chip Needed} = \frac{64}{4} = 16 \text{ RAM}$$

For memory Array Design



Address memory mapping

A ₁₅	A ₁₄	A ₁₃	A ₁₂	A ₁₁	A ₁₀	A ₉	A ₈	A ₇	A ₆	A ₅	A ₄	A ₃	A ₂	A ₁	A ₀	RAM chip	Address
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	RAM 0	0000H 0FFFH
0	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	RAM 1	1000H 1FFFH
0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0		
0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1		
1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	RAM 24	E000H EFFFH
1	1	1	0	1	1	1	1	1	1	1	1	1	1	1	1		
1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	RAM 15	F000H FFFFH
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1		

Table: Memory mapping

Ans:

For single RAM chip :

Address bus, $n = 10 \text{ bit}$

Data bus, $m = 8 \text{ bit}$

$$\begin{aligned}\text{Memory size of a single RAM chip} &= 2^n \times m \\ &= 2^{10} \times 8 \\ &= 1 \text{KB}\end{aligned}$$

We need 6KB RAM

So, number of RAM chip needed = 6

Memory Interfacing
Address Map

Memory:

It stores the binary codes for the sequence of instructions

Main memory Array Design

16 bit microprocessor can address 2^{16} locations
 $= 65536$ locations

 65536 Bytes
 $= 64 \text{ Kbytes}$

Main memory Array Design

3 Techniques ① Linear decoding

Microprocessor $\rightarrow 8 \text{ bit}$

① Data bus $\rightarrow 8 \text{ bit}$

② Address bus $\rightarrow 16 \text{ bit}$

RAM $\rightarrow 6 \text{ KB}$

RAM Address bus $\rightarrow 10 \text{ bit}$

RAM data bus $\rightarrow 8 \text{ bit}$

③ Draw the diagram using Linear decoding.

* For microprocessor

Address bus $n = 16 \text{ bit}$

Data bus $m = 8 \text{ bit}$

Microprocessor Capacity : $2^n \times m$
 $= 2^{16} \times 8$
 $= 64 \text{ KB}$

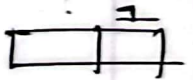
1.7_{10} = floating point representation

1. For integral part $1_{10} = 1_2$
For fractional part

$0.7 \times 2 = 1.4$	1
$.4 \times 2 = 0.8$	0
$.8 \times 2 = 1.6$	1
$.6 \times 2 = 1.2$	1
$.2 \times 2 = 0.4$	0
$.4 \times 2 = 0.8$	0
$.8 \times 2 = 1.6$	1
$.6 \times 2 = 1.2$	1

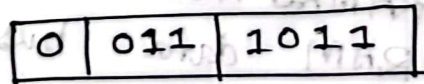
This is an endless process. We cannot represent this exactly as a floating point number. The closest we can come in four bits is 1.1011_2

$\therefore 1.7_{10} = 1.1011_2$



2. Normalize: $1.1011_2 \times 2^0 = 1.1011_2 \times 2^0$
3. Mantissa: 1011
4. Exponent: $0 + 3 = 3 = \text{011}_2$
5. Sign bits: 0

The result is



[This is not exact]

1. For fractional part

$$0.40625 \times 2 = 0.8125$$

$$0.8125 \times 2 = 1.625$$

$$0.625 \times 2 = 1.25$$

$$0.25 \times 2 = 0.5$$

$$0.5 \times 2 = 1.0$$

0
1
1
0
1

$$0.40625_{10} = 0.01101_2$$

Normalize : $0.01101_2 \times 2^0 = 1.101_2 \times 2^{-2}$

Mantissa : 1010

Exponent : $-2 + 3 = 1 = 1_2$

Sign bit = 0

sign exponent mantissa

0	001	1010
---	-----	------

8	4	2	1
1	1	1	0

-12.0 to 8 bit floating point

1 For integral part $12_{10} = 1100.0_2$

Normalize : $1100.0_2 \times 2^0 = 1.100_2 \times 2^3$

Mantissa : 1000

Exponent : $(3+3) = 6 = 100_2$

Sign = 1

1	100	1000
---	-----	------

① For integral part: $2_{10} = 10_2$

For fractional part:

$0.625 \times 2 = 1.25$	1	Generate 1 and continue
$0.25 \times 2 = 0.5$	0	Generate 0 and continue
$0.5 \times 2 = 1.0$	1	Generate 1 and nothing remain

$$\therefore 2.625_{10} = 10.101_2$$

2. Appending $\times 2^0$ to the end

2. Append an exponent part: $10.101_2 = 10.101_2 \times 2^0$

3. Normalize: $10.101_2 \times 2^0 = 1.0101_2 \times 2^1$

4. Mantissa: 0101

5. Exponent: $1+3 = 4 = 100_2$

6. Sign bit 0

The result: Sign exponent mantissa

$$\begin{array}{|c|c|c|} \hline 1 & 0 & 0 \\ \hline \end{array}$$

$$\begin{array}{|c|c|c|c|} \hline 0 & 1 & 0 & 1 \\ \hline \end{array}$$

$$(2^{1-1})$$

$$4 = 10$$

Conversion - 4.75

1. For integral part: $4_{10} = 100_2$

For fractional part:

$.75 \times 2 = 1.5$	1	Generate 1 and continue
$.5 \times 2 = 1.0$	1	Generate 1 and nothing remain

$$\therefore 4.75_{10} = 100.11_2$$

App Normalize $100.11_2 \times 2^0 = 1.0011 \times 2^2$

Mantissa
(0011)
Exponent (2+3)
= 5, 101

Exponent

0 1 0 1 0 1 0 1

$$0101010000 \mid 000010 = 2^{0.21875}_{10}$$

$$0.1010000 \times 10^2$$

$$0111 \overset{-8}{\overline{0011}} = 3$$

$$Q. \begin{array}{|c|c|c|c|c|} \hline 2 & 1 & .5 & 0.25 & 0.125 \\ \hline 1 & 0 & 1 & 0 & 1 \\ \hline \end{array}$$

$$0.111 \times 10^3$$

$$\begin{array}{r|rr} 4 & 2 & 2 \\ 1 & 1 & 1 \end{array} \bigg| 0 = 7_{10}$$

$$= 2.625_{10}$$

$$010101000000010_2 = 2.625_{10}$$

5

0 1 0 0 0 0 0 0 0 0

32	16	8	4	2	1
1	1	1	1	1	0

$= -32 + 30 = -2$

$$0.10000000 \times 10^{-2}$$

$$\begin{array}{r} 1.5125 \\ 0.001 \end{array} = 0.625_{10} \quad \text{125}_{10}$$

0110110000 | 000100

$$0.120120000 \times 10^4$$

$$\begin{array}{|c|c|c|c|c|} \hline 1 & 1 & 0 & 1 & 1 \\ \hline 8 & 4 & 2 & 1 & 5 \\ \hline \end{array}$$

$$= 23.5_{10}$$

0101000000

$$\boxed{0101000000} \times 10^{-1}$$

$$\begin{array}{r} 1.514125_{10} \times 25 \\ \hline 0.0101 \\ \hline = .3125_{10} \end{array}$$

~~3125~~

3. most significant bit - 1st bit value

1. Convert 0.40625 to our 8-bit floating point format.

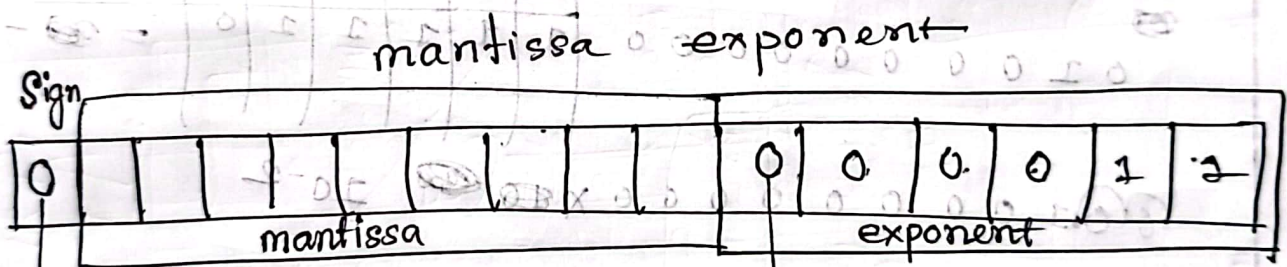
$$\begin{array}{rcl}
 0.40625 \times 2 & = & 0.8125 \\
 0.8125 \times 2 & = & 1.625 \\
 0.625 \times 2 & = & 1.25 \\
 0.25 \times 2 & = & 0.5 \\
 0.5 \times 2 & = & 1.0
 \end{array}$$

0	Generate 0 and con
1	Generate 1 and con
1	Generate 1 and
0	Generate 0 and con
1	Generate 1 and nothing remains

So, $0.40625_{10} = 0.01101_2$

2. Normalize $1.101_2 \times 2^{-2}$

3. Mantissa is 1010, exponent is -2+3



sign bit for mantissa

sign bit for exponent

$$0110100000 \mid \begin{array}{|c|c|c|c|c|c|} \hline 32 & 16 & 8 & 4 & 2 & 1 \\ \hline 0 & 0 & 0 & 0 & 1 & 1 \\ \hline \end{array} = 3$$

$$0110100000 \times 10^3$$

$$\begin{array}{|c|c|c|c|} \hline 4 & 2 & 1 & .5 \\ \hline 1 & 1 & 0 & .1 \\ \hline \end{array} = 6.5_{10}$$

$$0110100000_2 = 6.5_{10}$$

Convert 2.625 to our 8-bit floating point format

1. The integral part is easy, $2_{10} = 10_2$. For the fractional part:

$$0.625 \times 2 = 1.25$$

$$1.25 \times 2 = 0.5$$

$$0.5 \times 2 = 1.0$$

1
0
1

Generate 1 and continue
Generate 0 and continue
Generate 1 and nothing remains.

So $0.625_{10} = 0.101_2$ and $2.625_{10} = 10.101_2$

2. Add an exponent part: $10.101_2 = 10.101_2 \times 2^0$

3. Normalize: $10.101_2 \times 2^0 = 1.0101_2 \times 2^1$

4. Mantissa: 0101

5. Exponent: $1 + 3 = 4 = 100_2$

6. Sign bit is 0

The result is

0	100	0101
---	-----	------

Represented as hex, that is 45_{16}